

Final structure

Q	Lectures	Area	Marks	Type
1	01,02	class, object, constructor, getter/setter, overloading	20	Code
2	03	Noun/Verb Analysis + CRC Cards	15	Structured
3	06	Exception Handling + custom exceptions	15	Code
4	04	Relationships: inheritance, composition, aggregation, association, dependency	25	Code
5A	07	SOLID Principles	10	Structured
5B	05,08	abstract/interface/static + ArrayList	15	Code

Lecture 01 & 02 - OOP basics

Class	Blueprint/template for objects
Object	Instance of class, created with new
Attribute	Data/properties/state
Method	Behavior/function/action
Encapsulation	private attributes + public getters/setters
Abstraction	Show only needed details, hide unnecessary details

Access modifiers

public	accessible anywhere
private	same class only
protected	same package + subclasses
default	same package only

L01/02 - Class + constructor + getters/setters

```

class Product {
    private String productId;
    private String name;
    private double price;
    private int qty;

    Product() { // default constructor
        productId="N/A"; name="N/A"; price=0.0; qty=0;
    }
    Product(String productId, String name) { // overloaded
        this.productId = productId;
        this.name = name;
        this.price = 0.0; this.qty = 0;
    }
    Product(String productId, String name, double price, int qty) {
        this.productId = productId; this.name = name;
        setPrice(price); setQty(qty); // validation via setters
    }
    public String getProductId() { return productId; }
    public String getName() { return name; }
    public double getPrice() { return price; }
    public int getQty() { return qty; }
    public void setPrice(double price) {
        if(price > 0) this.price = price;
        else System.out.println("Invalid price");
    }
    public void setQty(int qty) {
        if(qty >= 0) this.qty = qty;
        else System.out.println("Invalid quantity");
    }
    public double totalValue() { return price * qty; }
    public void displayDetails() {
        System.out.println(productId+" "+name+" "+price+" "+qty);
    }
}

```

Constructor: same name as class, no return type, auto-called. **this.x** = current object's field.

Constructor overloading + method overloading

```

class Driver {
    private String id, name;
    private double salary;
    private int score;

    Driver() { id="N/A"; name="N/A"; salary=0; score=1; }
    Driver(String id, String name) { this.id=id; this.name=name; }
    Driver(String id, String name, double salary, int score) {
        this.id=id; this.name=name; this.salary=salary;
        setScore(score);
    }
    public void setScore(int score) {
        if(score >= 1 && score <= 5) this.score = score;
        else this.score = 1;
    }
    double calculateBonus() { return salary * 0.10; }
    double calculateBonus(double rate) { return salary * rate; }
    void display() { System.out.println(id+" "+name+" "+salary); }
}

```

Overloading Same method/constructor name, different parameter count/type/order. Return type alone does not count.

Lecture 03 - Noun/Verb Analysis Purpose: identify possible

classes, responsibilities and relationships from a requirement/story.

Process

1. Underline nouns/noun phrases = candidate classes/attributes.
2. Underline verbs = responsibilities/actions.
3. Reject weak nouns using rules.
4. Create final class list + CRC cards.

Noun rejection rules

Redundant	same meaning noun repeated
Event/operation	action/process, not object
Outside scope	not part of system
Meta-language	general word: system, information, details
Attribute	property of object: address, quantity, email

Typical answer format

Nouns: Customer, Order, Product, Quantity, Address, Payment, Cash, Card, Report, Manager, System
 Rejected:
 - Quantity, Address: attributes
 - Cash/Card: payment types
 - System: meta-language/general
 Final classes: Customer, Order, Product, Payment, Report

Lecture 03 - CRC Cards CRC = Class, Responsibility, Collaborator

Class	Responsibilities	Collaborators
Customer	place order, make payment	Order, Payment
Order	store items, calculate total, update status	Customer, Product, Payment
Product	store details, update stock	Order
Payment	process payment, validate amount	Order, Customer
Report	generate daily/weekly report	Order, Manager

Analysis classes: Entity = data objects, Boundary = user/system interaction, Control = flow/logic controller. Actors only login/use system - usually not classes unless stored/managed by system.

Lecture 04 - Relationship summary

Type	Meaning	Java code	UML
Inheritance	is-a	extends, super(), @Override	empty arrow
Composition	strong whole-part; part cannot exist without whole	new part inside constructor	filled diamond
Aggregation	weak whole-part; part can exist independently	part object passed as parameter	empty diamond
Association	has-a / uses-a persistent link	object as field/reference	plain line/arrow
Dependency	temporary use inside method	object as local variable/parameter only	dashed arrow

L04 - Inheritance + polymorphism

```

class Vehicle {
    protected String brand;
    protected double speed;
    Vehicle(String brand, double speed) {
        this.brand = brand; this.speed = speed;
    }
    public void start() { System.out.println("Vehicle starting"); }
    public void displayDetails() {
        System.out.println("Brand:"+brand+" Speed:"+speed);
    }
}
class Bus extends Vehicle {
    private int capacity;
    Bus(String brand, double speed, int capacity) {
        super(brand, speed); this.capacity = capacity;
    }
    @Override public void start() {
        System.out.println("Bus is starting...");
    }
    @Override public void displayDetails() {
        super.displayDetails();
        System.out.println("Passenger Capacity:"+capacity);
    }
}

```

Student ID :

Module Code :

L04 - Composition

```

class Engine {
    private String engineType;
    Engine(String engineType) { this.engineType = engineType; }
    public void displayEngine() {
        System.out.println("Engine Type: "+engineType);
    }
}
class Car extends Vehicle {
    private Engine engine; // part belongs to whole
    Car(String brand, double speed, String engineType) {
        super(brand, speed);
        this.engine = new Engine(engineType); // created inside
    }
    public void displayDetails() {
        super.displayDetails(); engine.displayEngine();
    }
}

```

Composition: whole owns part. If whole destroyed, part is destroyed.

L04 - Aggregation

```

class Driver {
    private String name, licenseNo;
    Driver(String name, String licenseNo) {
        this.name=name; this.licenseNo=licenseNo;
    }
    public void displayDriver() {
        System.out.println("Driver:"+name+" License:"+licenseNo);
    }
}
class Truck extends Vehicle {
    private Driver driver; // driver exists independently
    private double loadCapacity;
    Truck(String brand, double speed, Driver driver, double load) {
        super(brand, speed); // parent values
        this.driver = driver; // passed from outside
        this.loadCapacity = load;
    }
    @Override public void start() {
        System.out.println("Truck is starting...");
    }
    public void displayDetails() {
        super.displayDetails(); driver.displayDriver();
        System.out.println("Load Capacity:"+loadCapacity);
    }
}

```

L04 - Association + Dependency

```

// Association: persistent field reference
class Item { String name; double price; }
class Order {
    private Item item;
    private int qty;
    Order(Item item, int qty) { this.item=item; this.qty=qty; }
}
// Dependency: temporary use only
class Document { String title; Document(String title) {this.title=title; } }
class Printer {
    void printDocument(Document doc) { // not stored as field
        System.out.println("Printing: "+doc.title);
    }
}

```

L04 - Transport-style main pattern

```

public class TransportApp {
    public static void main(String[] args) {
        Driver d1 = new Driver("John", "DL123");
        Driver d2 = new Driver("Ann", "DL456");

        Bus bus = new Bus("Volvo", 88.0, 50);
        Truck truck = new Truck("Benz", 60.0, d2, 10.0);

        bus.start(); bus.displayDetails();
        truck.start(); truck.displayDetails();
    }
}

```

Keywords: **extends** inheritance, **super()** parent constructor, **@Override** redefine method, **protected** child access.

Core syntax dictionary

String compare	.equals() or .equalsIgnoreCase(), not ==	Scanner	next(), nextLine(), nextInt(), nextDouble()
main	public static void main(String[] args)	for-each	for(Type x : list)

Student ID :

Module Code :

Lecture 05 - Abstract class

Cannot instantiate	new Shape() invalid
Can have	abstract + concrete methods, fields, constructors
Child	must override all abstract methods or also be abstract

```
abstract class Shape {
    protected String color;
    Shape(String color) { this.color = color; }
    public abstract double getArea(); // no body
    public void display() { // concrete method
        System.out.println(color+" Area:"+getArea());
    }
}
class Circle extends Shape {
    private double radius;
    Circle(String color, double radius) {
        super(color); this.radius = radius;
    }
    @Override public double getArea() {
        return Math.PI * radius * radius;
    }
}
```

Lecture 05 - Interface

Interface	contract; methods public abstract by default
Fields	public static final by default
Multiple	class can implement many interfaces

```
interface Payable { double calculatePayment(); }
interface Reportable { void generateReport(); }

class Freelancer implements Payable, Reportable {
    private String name; private double hours, rate;
    Freelancer(String name, double hours, double rate) {
        this.name=name; this.hours=hours; this.rate=rate;
    }
    @Override public double calculatePayment() { return
        hours * rate; }
    @Override public void generateReport() {
        System.out.println(name+"
        Payment:"+calculatePayment());
    }
}
```

Lecture 05 - Static members

```
class Student {
    private String id, name;
    static String universityName; // one copy shared by all
    static int count = 0;
    static { universityName = "SLIIT"; } // runs once
    Student(String id, String name) {
        this.id=id; this.name=name; count++;
    }
    static void displayUniversity() {
        System.out.println(universityName);
    }
    void displayStudentInfo() {
        System.out.println(id+" "+name+" "+universityName);
    }
}
// use
Student.universityName = "SLIIT";
Student.displayUniversity();
Student s1 = new Student("S001", "Kamal");
System.out.println(Student.count);
```

Static method: cannot use this or directly access non-static fields.

Lecture 06 - Exception Handling

Exception	runtime abnormal event that disturbs normal flow
Error hierarchy	Throwable -> Error / Exception -> RuntimeException
Checked	must catch/declare: IOException, SQLException
Unchecked	RuntimeException: Arithmetic, NullPointerException, ArrayIndexOutOfBoundsException

```
try {
    int ans = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Error: " + e.getMessage());
} finally {
    System.out.println("Always runs");
}
```

try	risky code	catch	handle exception
finally	always executes	throw	manually send exception
throws	method declares it may throw	getMessage()	returns message

L06 - Custom exception template

```
class InvalidUserNameException extends Exception {
    public InvalidUserNameException(String message) {
        super(message);
    }
}
class InvalidPasswordException extends Exception {
    public InvalidPasswordException(String message) {
        super(message);
    }
}
public class DemoApp {
    public static void main(String[] args) {
        String username = "abc";
        String password = "12345";
        try {
            if(username.length() < 6) {
                throw new InvalidUserNameException(
                    "Username must be at least 6 characters");
            }
            if(password.length() < 8) {
                throw new InvalidPasswordException(
                    "Password must be at least 8 characters");
            }
            System.out.println("Validation successful");
        } catch (InvalidUserNameException e) {
            System.out.println(e.getMessage());
        } catch (InvalidPasswordException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

age invalid	age < 18
marks invalid	marks < 0 marks > 100
email invalid	!email.contains("@")
fee invalid	fee < 5000

Lecture 08 - ArrayList operations

Create	ArrayList<Book> books = new ArrayList<>();
Add	books.add(new Book(isbn,title));
Access	books.get(i)
Modify	books.set(i, obj)
Remove	books.remove(i) or remove(obj)
Count	books.size(), books.isEmpty()

```
import java.util.ArrayList;
class Book {
    private String isbn, title;
    Book(String isbn, String title) { this.isbn=isbn;
        this.title=title; }
    public String getIsbn() { return isbn; }
    public void displayDetails() { System.out.println(isbn+"
        "+title); }
}
```

L08 - Input until -99 + remove

```
Scanner input = new Scanner(System.in);
ArrayList<Book> books = new ArrayList<>();

System.out.print("Enter ISBN: ");
String isbn = input.nextLine();
while(!isbn.equals("-99")) {
    System.out.print("Enter title: ");
    String title = input.nextLine();
    books.add(new Book(isbn, title));

    System.out.print("Enter ISBN: ");
    isbn = input.nextLine();
}

System.out.print("Enter ISBN to remove: ");
String searchIsbn = input.nextLine();
boolean found = false;
for(int i=0; i<books.size(); i++) {
    if(books.get(i).getIsbn().equals(searchIsbn)) {
        books.remove(i);
        found = true;
        System.out.println("Book removed successfully");
        break;
    }
}
if(!found) System.out.println("ISBN not found");

System.out.println("Total: "+books.size());
for(Book book : books) {
    book.displayDetails();
}
```

Generics: <Book> type-safe. Primitive wrappers: int -> Integer, double -> Double, char -> Character, boolean -> Boolean.

Lecture 07 - SOLID Principles

Letter	Principle	Definition	Benefit
S	Single Responsibility	one class has one responsibility / one reason to change	maintain, test
O	Open/Closed	open for extension, closed for modification	extend safely
L	Liskov Substitution	subclass can replace superclass safely	safe inheritance
I	Interface Segregation	do not force unused methods; use small interfaces	clean interfaces
D	Dependency Inversion	depend on abstractions, not concrete classes	loose coupling

Design decay symptoms: Rigidity = hard to change, Fragility = changes break other parts, Immobility = hard to reuse, Viscosity = hacks easier than proper design, Opacity = hard to understand.

SOLID compact examples

```
// SRP bad: one class does many jobs
class UserService { void register(){} void sendEmail(){} void log(){} }
// SRP good: UserService, EmailService, Logger separated

// OCP good
interface Payment { void pay(); }
class CardPayment implements Payment { public void pay(){} }
class CashPayment implements Payment { public void pay(){} }

// ISP good
interface Printable { void print(); }
interface Scannable { void scan(); }
class BasicPrinter implements Printable { public void print(){} }

// DIP good
interface Database { void insert(); }
class MySQLDatabase implements Database { public void insert(){} }
class OrderService {
    private Database db;
    OrderService(Database db) { this.db = db; }
    void saveOrder() { db.insert(); }
}
```

Common compile-safe notes

- Only one **public** class per .java file; other helper classes can be non-public.
- Constructor name must exactly match class name.
- String compare: name.equals("Kamal").
- When removing from ArrayList by ID, compare getter value: obj.getId().equals(searchId).
- After nextInt()/nextDouble(), use extra input.nextLine() before reading a full line.
- public static void main(String[] args) is required to run.